

HDOC – Experiment 32

Question

Write a C program to perform the following menu-driven operations using separate functions:

1. Get two variable length 2D arrays and prove that matrix addition is commutative.
2. Get two variable length 2D arrays and prove that matrix multiplication is not commutative.
3. Exit

❖ Algorithm

Main Algorithm

1. Start.
2. Display the menu:
 - Matrix addition — prove commutative property
 - Matrix multiplication — demonstrate non-commutative property
 - Exit
3. Read the user's choice.
4. Use a switch statement:
 - If choice is 1, call `additionCommutative()`.
 - If choice is 2, call `multiplicationNonCommutative()`.
 - If choice is 3, terminate the program.
 - Otherwise, display "Invalid choice".
5. Repeat the menu until the user chooses 3.

6. Stop.

Algorithm for Matrix Addition

1. Read the number of rows and columns.
2. Ensure both are positive.
3. Declare two VLA matrices A and B.
4. Read the elements of both matrices.
5. Calculate $A + B$.
6. Calculate $B + A$.
7. Display both results.
8. Compare corresponding elements.
9. If all elements are equal, display that matrix addition is commutative.

Algorithm for Matrix Multiplication

1. Read the order n of the two square matrices.
2. Ensure n is positive.
3. Declare two $n \times n$ VLA matrices A and B.
4. Read their elements.
5. Calculate $A \times B$.
6. Calculate $B \times A$.
7. Display both products.
8. Compare corresponding elements.
9. If they differ, show that multiplication is not commutative for the entered matrices.
10. If they happen to be equal, explain that these particular matrices commute, although matrix multiplication is **not commutative in general**.

❖ Test case table

Test	Operation	Input	Expected Result
1	Addition	$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$A+B = B+A = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$
2	Addition	$A = \begin{bmatrix} 1 & -2 \\ 0 & 4 \end{bmatrix}, B = \begin{bmatrix} 3 & 5 \\ 2 & -1 \end{bmatrix}$	Both sums are equal
3	Multiplication	$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$AB \neq BA$
4	Multiplication	A = identity matrix, B = any 2×2 matrix	$AB = BA$; matrices commute in this particular case
5	Validation	Rows/columns/order = 0 or negative	Ask for valid positive dimensions again
6	Menu	Choice = 7	Display "Invalid choice!"
7	Exit	Choice = 3	Program terminates

❖ Program code in C

```
#include <stdio.h>
```

```
void readMatrix(int rows, int cols, int matrix[rows][cols]);
```

```
void printMatrix(int rows, int cols, int matrix[rows][cols]);
```

```
void additionCommutative(void);
```

```
void multiplicationNonCommutative(void);
```

```
int main(void)
```

```
{
```

```
    int choice;
```

```
do
{
    printf("\n===== MATRIX OPERATIONS =====\n");
    printf("1. Prove matrix addition is commutative\n");
    printf("2. Prove matrix multiplication is not commutative\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            additionCommutative();
            break;

        case 2:
            multiplicationNonCommutative();
            break;

        case 3:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice! Please enter 1, 2 or 3.\n");
    }

} while (choice != 3);
```

```
    return 0;
}
```

```
/* Function to read a matrix */
```

```
void readMatrix(int rows, int cols, int matrix[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
}
```

```
/* Function to display a matrix */
```

```
void printMatrix(int rows, int cols, int matrix[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
```

```

    {
        printf("%d\t", matrix[i][j]);
    }
    printf("\n");
}
}

```

/* Function to prove $A + B = B + A$ */

void additionCommutative(void)

```

{
    int rows, cols;

    int i, j;

    int equal = 1;

    do
    {
        printf("\nEnter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);

        if (rows <= 0 || cols <= 0)
            printf("Rows and columns must be positive.\n");

    } while (rows <= 0 || cols <= 0);

    /* Variable Length Arrays */
    int A[rows][cols];
    int B[rows][cols];

```

```

int AB[rows][cols];

int BA[rows][cols];


printf("\nEnter elements of matrix A:\n");

readMatrix(rows, cols, A);


printf("\nEnter elements of matrix B:\n");

readMatrix(rows, cols, B);


/* Calculate A + B and B + A */
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        AB[i][j] = A[i][j] + B[i][j];
        BA[i][j] = B[i][j] + A[i][j];
    }
}


printf("\nA + B:\n");

printMatrix(rows, cols, AB);


printf("\nB + A:\n");

printMatrix(rows, cols, BA);


/* Compare the matrices */
for (i = 0; i < rows; i++)
{

```

```

    for (j = 0; j < cols; j++)
    {
        if (AB[i][j] != BA[i][j])
        {
            equal = 0;
        }
    }
}

if (equal)
{
    printf("\nA + B = B + A\n");
    printf("Hence, matrix addition is COMMUTATIVE.\n");
}
else
{
    printf("\nA + B != B + A\n");
}
}

```

```

/* Function to demonstrate AB != BA */
void multiplicationNonCommutative(void)
{
    int n;
    int i, j, k;
    int equal = 1;

```

```

do
{
    printf("\nEnter the order of the square matrices: ");
    scanf("%d", &n);

    if (n <= 0)
        printf("Matrix order must be positive.\n");

} while (n <= 0);

/* Variable Length Arrays */
int A[n][n];
int B[n][n];
int AB[n][n];
int BA[n][n];

printf("\nEnter elements of matrix A:\n");
readMatrix(n, n, A);

printf("\nEnter elements of matrix B:\n");
readMatrix(n, n, B);

/* Calculate A x B */
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        AB[i][j] = 0;
    }
}

```

```
    for (k = 0; k < n; k++)
    {
        AB[i][j] += A[i][k] * B[k][j];
    }
}
}
```

```
/* Calculate B x A */
```

```
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        BA[i][j] = 0;

        for (k = 0; k < n; k++)
        {
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}
```

```
printf("\nA x B:\n");
printMatrix(n, n, AB);
```

```
printf("\nB x A:\n");
printMatrix(n, n, BA);
```

```

/* Compare AB and BA */
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        if (AB[i][j] != BA[i][j])
        {
            equal = 0;
        }
    }
}

if (!equal)
{
    printf("\nA x B != B x A\n");
    printf("Hence, matrix multiplication is NOT COMMUTATIVE.\n");
}
else
{
    printf("\nA x B = B x A for these particular matrices.\n");
    printf("These matrices commute, but matrix multiplication ");
    printf("is not commutative in general.\n");
}
}

```

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp32]
$ vi array5.c

(kali㉿kali)-[~/Desktop/exp32]
$ gcc array5.c -o array5

(kali㉿kali)-[~/Desktop/exp32]
$ ./array5

===== MATRIX OPERATIONS =====
1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit
Enter your choice: 1

Enter number of rows and columns: 2 2

Enter elements of matrix A:
1 2 3 4

Enter elements of matrix B:
5 6 7 8

A + B:
6      8
10     12

B + A:
6      8
10     12

A + B = B + A
Hence, matrix addition is COMMUTATIVE.

===== MATRIX OPERATIONS =====
1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit
Enter your choice: 1
```

Enter number of rows and columns: 2 2

Enter elements of matrix A:

1 -2 0 4

Enter elements of matrix B:

3 5 2 -1

A + B:

4 3
2 3

B + A:

4 3
2 3

A + B = B + A

Hence, matrix addition is COMMUTATIVE.

===== MATRIX OPERATIONS =====

1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit

Enter your choice: 2

Enter the order of the square matrices: 2 2

Enter elements of matrix A:

1 2 3 4

Enter elements of matrix B:

5 6 7 8

A x B:

14 17
26 31

B x A:

B x A:

18 19
26 27

$A \times B \neq B \times A$

Hence, matrix multiplication is NOT COMMUTATIVE.

===== MATRIX OPERATIONS =====

1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit

Enter your choice: Invalid choice! Please enter 1, 2 or 3.

===== MATRIX OPERATIONS =====

1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit

Enter your choice: 1

Enter number of rows and columns: 0 -2
Rows and columns must be positive.

Enter number of rows and columns: 2 2

Enter elements of matrix A:

1 2 3 4

Enter elements of matrix B:

4 3 2 1

A + B:

5 5
5 5

B + A:

5 5
5 5

A + B = B + A

$A + B = B + A$

Hence, matrix addition is COMMUTATIVE.

===== MATRIX OPERATIONS =====

1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit

Enter your choice: 7

Invalid choice! Please enter 1, 2 or 3.

===== MATRIX OPERATIONS =====

1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit

Enter your choice: 3

Exiting program...

(kali@kali)-[~/Desktop/exp32]
\$